What a Typeclass Owes: Obligations of a Library Typeclass Encoding, from a Formal Model of Overload Resolution

Document #: DXXXXR0
Date: 2026-10-17
Project: Programming Language C++
Audience: LEWG, EWG
Reply-to: Steve Downey
<sdowney@gmail.com>

Contents

1 Abstract **1**

2 Introduction **2**

3 The encoding **2**

4 The model **3**

5 Why the design works: a variable cannot be overloaded **3**

6 Obligations **4**
6.1 For the typeclass author 4
6.2 For the instance author 5
6.3 For users and the authors of functions on instances 5

7 Inheritance between instance types **5**

8 Across translation units **6**

9 Results about dispatch idioms **7**

10 Findings about the Standard and implementations **8**
10.1 The Standard 8
10.2 Implementations 8

11 Limits of the model **8**

12 Implementation experience **9**

13 References **9**

1 Abstract

Typeclasses can be written in C++ as a library. An instance is a partial specialization of a variable template whose primary is `std::false_type`, and a user calls the instance's members. Each operation is also an object whose call operator finds the instance through a defaulted constant template parameter. beman.transpose [beman.transpose] is built this way. This paper reports what a formal model of the Standard's selection rules says about that encoding: what it guarantees by construction, what it leaves as an obligation on the typeclass author, the instance author and the user, and which of those obligations no compiler can check.

The model is written in Lean 4. It transcribes template argument deduction, substitution failure, constraints and subsumption, Mandates, partial ordering, partial specialization matching and overload resolution, each rule cited by stable name and paragraph against the working draft [N5054]. Its predictions were checked against GCC 13 to 16 and Clang 17 to 24 on 91 generated programs.

The short form of the result is that the design is sound because it puts a variable everywhere C++ would otherwise build an overload set, and its hazards appear exactly where an overload set comes back.

2 Introduction

A typeclass names a set of operations and the laws they obey, and lets each type supply them separately from the type's definition. Haskell selects the implementation by type; C++ has no such mechanism in the language, but it has three that can be combined to the same effect: partial specialization matching to pick an instance for a type, default template arguments to look it up at a call, and constraints to make a call with no instance not viable.

The questions this paper answers are the ones a designer has to answer before recommending the technique:

— Is instance selection coherent: does a type get at most one instance, and never a silent choice between two?
— Is it stable: does adding an instance somewhere change only the calls it should change?
— What must each party do so that these hold, and which of those duties does a compiler check?

The answers come from proofs about a model of the rules, not from experiments with one compiler. Where a compiler disagrees with the text, the disagreement is reported against the cited sentence.

3 The encoding

A typeclass has two layers.

The instance is the bundle of operations, and it is the interface a user calls. A lookup variable template names it, with `std::false_type` as the instance for types that have none:

```
template <class T>
inline constexpr auto functor = std::false_type{};

template <class V>
inline constexpr auto functor<std::optional<V>> = OptionalFunctorMap<V>{};
```

The instance's operations are its members. A user names the instance of record and calls them:

```
const auto &app = applicative<std::optional<int>>;
auto sum = app.invoke(sum3, std::optional{1}, std::optional{2},
                      std::optional{3});
```

An instance is usually built from an implementation that supplies a minimal basis, through a derivation base that supplies the rest. The derivation base takes the implementation as a protected base, and the instance re-exposes the basis with using-declarations:

```
template <class V>
struct optional_applicative_map
    : derive_applicative<optional_applicative_impl<V>> {
    using optional_applicative_impl<V>::invoke;
    using optional_applicative_impl<V>::pure;
};
```

Each named operation is also an operation object of its own, a second interface over the first, with one call operator. The call operator finds the instance through a trailing constant template parameter, a reference defaulted to the lookup, and is constrained on the call through it:

```
struct fmap_fn {
    template <class F, class T,
              const auto &TC = functor<std::remove_cvref_t<T>>>
        requires requires { TC.fmap(std::declval<F>(), std::declval<T>()); }
    constexpr auto operator()(F &&f, T &&t) const {
        return TC.fmap(std::forward<F>(f), std::forward<T>(t));
    }
};
inline constexpr fmap_fn fmap{};
```

The operation object is shaped like a customization point object, but it does not dispatch through argument-dependent lookup or `tag_invoke`. It dispatches through the lookup variable.

Two other ways to supply an instance exist beside the canonical lookup. A caller can name an instance object and call its member, `tc.fmap(f, v)`. And an operation can take the instance as a trailing by-value argument whose type defaults to the type of the looked-up instance, as `traverse` does with its policy.

4 The model

The model gives each mechanism twice: as an inductive relation whose constructors transcribe the Standard's sentences, each cited, and as an executable function, with a proof that the two agree. Every headline theorem is checked to rest on no unproven assumption.

Where the Standard needs the meaning of an expression the model does not model, such as whether a concept is satisfied, an oracle answers, and the oracle says whether a failure is in the immediate context. Programs are lists of declarations, so "declared before use" is a property the model can state.

The executable model emits each test case as a C++ translation unit whose `static_assert`s encode the model's expectation, and the case is compiled with each compiler. The expectation is always what the text says. A compiler that disagrees is reported, never outvoted.

5 Why the design works: a variable cannot be overloaded

The encoding puts a variable at each point where C++ would otherwise build an overload set, and most of its good behaviour follows.

The operation object. `fmap` is a variable. Name lookup for `fmap(f, x)` finds one entity, so the call has one candidate, the object's call operator. Argument-dependent lookup does not run when ordinary lookup finds an object, and nobody can add an overload that competes with it.

The instance. `functor<T>` is a variable template. Choosing an instance is partial specialization matching, not overload resolution, so conversion ranking, argument-dependent lookup and the preference for non-templates never touch it. The model proves:

— *Coherence.* For a type, lookup selects the most specialized matching instance, or reports ambiguity, or finds none and yields the primary. It never chooses silently between unordered instances.
— *Stability.* Declaring an instance moves lookup only at types the new instance matches, and there only to the new instance or to ambiguity. Lookups of other typeclasses do not move.
— *Separability.* For any two conversion rankings, if both select a candidate at a position, it is the same candidate with the same template arguments. The ranking decides which candidate wins; it cannot change the instance a candidate carries.

The call. At the call surface the same results hold for the operation object. Deducing it follows lookup at the argument's type with cv and reference qualifiers removed. It succeeds when an instance is selected, fails softly, making the call not viable, when none is, and fails hard only when lookup is ambiguous. Declaring an instance

leaves a call unchanged unless the instance matches the call's key. When it does, the call becomes viable through it, or is ill-formed by ambiguity.

The members. With an instance in hand, `app.invoke(...)` or `TC.fmap(...)` is member lookup in one class, which nothing at namespace scope can join. Argument-dependent lookup applies only to an unqualified name ([basic.lookup.argdep]/1), and member lookup stops at the first class whose scope declares the name ([class.member.lookup]/3). The instance's using-declaration of its implementation's `invoke` therefore hides the derivation base's, and the call has one template. The model does not resolve this call. It takes the member's answer as given and proves that the operation object passes it on unchanged: viable where the member call is valid, not viable where it is invalid, and ill-formed where checking it is a hard error. The operation object can add no accuracy the member lacks. That is the one place where the design relies on its authors more than on its shape, and the first obligation below on the typeclass author is what makes it hold.

Every hazard the model found is a place where an overload set comes back.

6 Obligations

Each obligation below is a hypothesis some theorem needs that the language does not guarantee. Each is classed by how it is discharged: by construction, compiler-checked, library-checkable, or undiagnosable.

6.1 For the typeclass author

— **The members are accurate.** Each member an instance exposes must be constrained so that a probe of it fails softly where the call is not viable, and succeeds only where it is. The operation object's constraint is the member call, so it can be no more accurate than the member. A derived member with no constraint and a deduced return type fails this: a probe instantiates its body to deduce the type, and an error there is not in the immediate context ([dcl.spec.auto.general]/15). So does a constraint with an alternative weaker than the body it selects. A probe of every typeclass in beman.transpose found members of each kind in every derivation base, and two members that are wrong about the call, not only about the probe. `any_of` with a predicate that cannot be called reports the call as valid, and the call fails. The optional Monad's `bind` accepts a function that does not return an optional, and returns a plain value. Where an implementation supplies only `ap`, the derived `invoke` curries the function. It accepts a function that cannot take the arguments and returns an unsaturated partial application in the context. *Not by construction.*

— **Instances of std types live with the typeclass.** An instance must be visible wherever its lookup is used on its type. For a library type the type's own header is enough, since every use has to include it. For a std type the only header every use is guaranteed to include is the typeclass's. An instance in a header of its own, as beman.transpose declares the instances for `std::array`, `std::expected`, `std::simd::basic_vec` and, for folding and traversal, `std::vector`, makes a program that includes it late, or in some translation units and not others, ill-formed with no diagnostic required. GCC diagnoses the late case. Clang keeps the first answer, and the instance stays invisible after its header is included. This is Haskell's orphan rule with a well-formedness consequence. *Undiagnosable.*

— **The primary is soft.** The lookup's primary must be a value such as `std::false_type`, not a `static_assert`. A default template argument is substituted before constraints are checked, so a requires-clause on the same template cannot guard it. A hard primary makes every call ill-formed whose deduction reaches the lookup, including calls that would have chosen another overload. *By construction in beman.transpose.*

— **The default is shapeless.** A missing instance makes the call not viable only because `std::false_type` has no `fmap`. *Library-checkable: one `static_assert` per typeclass that the primary fails the shape.*

— **The lookup key is normalized.** The lookup is at `std::remove_cvref_t<T>`, so every spelling of a type reaches one instance. *By construction.*

— **No other candidate reaches the lookup.** Stability at the call surface needs the call's other candidates not to look the instance up. An operation object with one call operator meets this by construction, since a variable cannot be overloaded. One with several, as beman.transpose's `fold_map` and `combine_all` have, meets it when they differ by arity and each puts the lookup default after a parameter a shorter call cannot deduce: that overload fails before it reaches the lookup. With the default first, it reaches the lookup in a

call meant for another overload, and an ambiguous lookup makes that call ill-formed. If a named operation were a function template or an overload set, anyone could add an overload, and each added overload would have to stay away from the lookup.

— **Refinement is visible.** Write a refinement between conditions as constraints, so subsumption can see it. An `enable_if` encoding of the same condition has the same viable set, and loses every ordering a requires-clause would win, without a diagnostic. *Convention.*

6.2 For the instance author

— **Canonicity.** No two instances may match a type with neither more specialized. *Compiler-checked under the text*, at the lookup and at every call that substitutes it. At the call surface the compilers do not check it; see Implementations.

— **Expose the accurate member.** A using-declaration of an implementation's member hides the derivation base's member of that name. That is right when the implementation's member is the accurate one, as for every Applicative `invoke` in beman.transpose, and wrong when it is not: the optional and vector Functor implementations' `fmap` is unconstrained, and re-exposing it hides `derive_functor`'s constrained `fmap`. A call with a function that cannot take the element is then a hard error, through the member call and through the operation object. *Convention.*

— **Visibility.** Every instance must be declared before any use of its lookup, the same in every translation unit. *Undiagnosable*; see Across translation units.

— **Distinct instance types.** The by-value route identifies an instance by its type alone, so two instances of one typeclass must be objects of different types. *Convention.*

— **Superclass coherence.** A Functor derived from a stronger instance, as a Monad's `as_functor` returns, should behave as the canonical Functor where one exists. The two are different template arguments, so they can agree only in behaviour. *A law; undiagnosable, and the program is well-formed either way.*

6.3 For users and the authors of functions on instances

— **Exact-type use.** A signature should take an instance at its deduced type and never name an instance type, not even as a base such as `derive_functor<I>`, nor an implementation an instance inherits through a protected base. See Inheritance between instance types. *Undiagnosable: the program is well-formed and calls what the author did not mean.*

— **Soft failure for pinning functions.** A function that takes an instance explicitly and has no constraint is viable for a type with no instance, and fails in its body. Whoever writes such a function owes the constraint.

7 Inheritance between instance types

beman.transpose relates no two instance types by public inheritance, but inheritance is a choice an implementer can make, and the paper treats it as allowed. Forbidding it would need a means of enforcement that does not exist. The question is what it costs.

Within one discipline it costs nothing. Replacing derivation between instance types by composition changes no call that uses every affected class at its exact type and never deduces through it as a base. The model proves this for its fragment. Outside that discipline there are five hazards. The first, second and fifth are confirmed on every compiler tested; the third is a theorem about identity; the fourth follows C++26 and is where Clang and GCC currently differ.

Ranking beats refinement. A derived-to-base conversion has Conversion rank, and conversions are compared before the preference for non-templates or partial ordering.

```
struct functor_i {};
struct applicative_i : functor_i {};
struct monad_i : applicative_i {};
```

```
template <class T> void use(const T&);   // #1
void use(const applicative_i&);          // #2

use(applicative_i{});  // #2: conversions tie, the non-template is preferred
use(monad_i{});        // #1: an exact match beats derived-to-base
```

Deriving one instance type from another moved the call to the less refined overload.

Deduction from a derived class. A parameter of class template form deduces from a class derived from it ([temp.deduct.call]/4).

```
template <class I> struct derive_functor {};
struct opt_impl {};
struct opt_monad_map : derive_functor<opt_impl> {};

template <class I> void use(const derive_functor<I>&);
use(opt_monad_map{});   // deduces I = opt_impl
```

A Monad instance silently matches a Functor-shaped signature.

Two paths to the superclass. Slicing an instance to its base gives one Functor and lookup gives another, and they are different template arguments.

Constant-parameter slicing. A derived instance passed as a constant template argument for a base-typed parameter is sliced and identified with the base value.

A protected base does not hide the implementation. `derive_functor<Impl>` inherits `Impl` protected, which reads as keeping it out of view. Implicit conversion sequences ignore accessibility ([over.best.ics.general]/2), and deduction from a derived class goes through the protected base, so a function naming `Impl` is selected, and then the conversion makes the call ill-formed ([conv.ptr]/3, [dcl.init.ref]/4).

```
struct impl {};
struct derive_impl : protected impl {};
struct map : derive_impl {};

void use(const impl&);
use(map{});   // selected, then ill-formed: impl is an inaccessible base
```

The call is not merely not viable, so no other overload is tried. Inside a requires-expression the same failure is part of substitution and the requirement is simply false.

All five come down to the exact-type rule above: inherit if you like, but never name an instance type, or what it is built from, in a signature.

8 Across translation units

Some violations make a program ill-formed with no diagnostic required, because nothing useful can be said about one entity with two contrary definitions. The model follows the Standard's lead. It treats a program as a list of translation units, each resolved on its own, and when units disagree it gives a verdict, ill-formed with no diagnostic required, naming the rule and the use, and says nothing more.

A use of a lookup gets a verdict when its selection is not the one it would make with every instance of the program reachable:

— an instance declared later in the same unit gives the specialization a second point of instantiation with a different meaning ([temp.point]/7);
— an instance declared only in another unit is not reachable from a use that would make use of it ([temp.spec.partial.general]/1).

For a single unit, having no verdict is exactly the condition that every instance is declared before every use of its lookup. Stated per unit, stability at the call surface says a call sees what precedes it: an instance declared after the call cannot move it, and can only make the program ill-formed with no diagnostic required.

The important result is about identity. One might expect a function whose behaviour depends on an instance to be safe across units whenever the instance is one of its template arguments, since units that select different instances would then have different specializations. The operation object does take the instance as a template argument, but as a reference to the lookup variable, `const auto &TC = functor<...>`. A reference argument is the entity it designates, and `functor<box<int>>` is one entity in every unit, whatever its initializer selects. Two units that select different instances therefore deduce *one* specialization, and the program gives one function two meanings, ill-formed with no diagnostic required by [basic.def.odr]/16. Pinning names the instance object and does separate the units. The by-value policy names the instance's type and separates them only when instances have distinct types.

Being a variable is what makes the operation object immune to overloading, and what exposes it to the one-definition rule. Visibility is the obligation that keeps that exposure from mattering.

9 Results about dispatch idioms

The model was used to compare idioms a typeclass author is likely to reach for.

Mandates. A Mandate, checked only after selection, never changes which candidate is selected; it can only turn a selection into an ill-formed program, with no fallback to another overload. Turning a Mandate into a constraint does remove the candidate where the Mandate fails. Where it holds, the larger constraint can change partial ordering:

```
template <class T> requires class_like<T> void g(T);              // #1
template <class T> void g(T) { static_assert(empty_like<T>); }    // #2

g(X{});   // #1, the more constrained; #2's Mandate is never checked
```

Promote #2's Mandate to `requires empty_like<T>`, and neither constraint subsumes the other: the call becomes ambiguous, an outcome neither the old resolution nor removing #2 gives. So promotion is not merely removal.

`enable_if` and requires-clauses. With the same condition, an `enable_if` in a trailing return type and a requires-clause give the same viable set, and the same resolution wherever ordering does not consult subsumption. They differ in two ways. A requires-clause wins orderings by subsumption that `enable_if` cannot. And the step order differs: a requires-clause is checked before the function type is substituted, so it shields a hard error in a parameter's type, while `enable_if` in the return type is checked only as that type is substituted, after the parameters:

```
template <class T> requires integral_like<T>
void f(T, std::remove_cvref_t<decltype(lookup<T>)>);                // not viable
template <class T>
auto f(T, std::remove_cvref_t<decltype(lookup<T>)>) -> std::enable_if_t<integral_like<T>>;
                                                                    // hard error
```

where `lookup<T>` fails hard for the argument's type. Substitution follows lexical order ([temp.deduct.general]/7), so a leading return type would check `enable_if` first; the result above is specific to the trailing form.

Dispatch styles. For an ordered list of conditions, constraint exclusion, priority tags and an `if constexpr` chain select the same candidate. With no condition holding, the first two leave the call not viable and the chain is ill-formed. They also differ in which conditions are evaluated. The chain stops at the first that holds, and so does exclusion written with the negations first, since a conjunction is checked left to right. Priority tags check every overload's condition, because every overload is a candidate, so a later condition that fails hard breaks the call. The library's lookups default to `std::false_type`, so its own conditions cannot fail that way.

10 Findings about the Standard and implementations

10.1 The Standard

— [temp.constr.order]/2, a note, says subsumption "defines a partial ordering on constraints". It is reflexive and transitive, but `A ∧ B` and `B ∧ A` subsume each other and are different constraints, so it is a preorder. It is a partial order on constraints taken up to mutual subsumption, which is presumably what is meant. Normative text is unaffected.

10.2 Implementations

— **An ambiguous instance is treated as no instance.** At a type two instances match with neither more specialized, the use of the lookup in the operation object's default template argument is ill-formed by [temp.spec.partial.match]/1, and since it is an instantiation it is outside the immediate context ([temp.deduct.general]/8), so the call is ill-formed. GCC 13 to 16 and Clang 17 to 24 instead treat the call as a deduction failure and select another overload if there is one. The same holds for the by-value route. So canonicity is not checked at the call surface in practice: overlapping instances make a type silently look as if it had none. The compilers do diagnose the lookup itself.

— **Constraints before the function type.** Clang 17 to 20 substitute the function type, and check conversions to non-dependent parameters, before checking constraints, the ordering CWG2369 [CWG2369] replaced. The draft's own second example in [temp.deduct.general]/5 fails to compile on them, and a requires-clause does not shield a hard error in a parameter's type. Clang 21 to 24 and GCC 13 to 16 follow the current text.

— **A perfect non-template match.** GCC skips deduction for template candidates when a non-template is an exact match, in some cases, and Clang 21 to 24 do so in more, including constraint satisfaction. This is the direction of P3606 [P3606R2]. The two implementations disagree about what it covers, which is evidence for what its wording needs to say.

— **Placeholder type-constraints.** [temp.over.link]/6 makes constant template parameters equivalent ignoring type-constraints on placeholder types, so `auto` and `shape auto` parameters leave the tie-break of [temp.func.order]/8 open. Clang 19 to 24 and GCC 16 agree. GCC 13 to 15 and Clang 17 and 18 report the call ambiguous.

— **A derived value for a base-typed constant parameter.** Under [temp.arg.nontype] as rewritten by P2308R1 [P2308R1], a derived class value is accepted for a constant template parameter of base class type, sliced, and identified with the equal base value. Clang does this. GCC 13 rejects it as not a converted constant expression, the rule before P2308R1.

11 Limits of the model

The model covers a fragment. Partial ordering is proved transitive for templates of type parameters whose parameter types have no references; with references, the tie-breaks of [temp.deduct.partial]/9 enter, and the evidence is a search over 5832 triples of parameter shapes that found no counterexample. Deduction is proved sound and most general on the first-order fragment; references nested in a parameter type have two solutions under reference collapsing, and deduction gives the one the form `T&` prescribes. Dispatch styles are compared exhaustively over three conditions, not proved for every length. The conversion ranking covers identity, derived-to-base and reference binding; arithmetic conversions, user-defined conversions and access from inside a class are outside. Concept satisfaction is an oracle, not a semantics of expressions. A registry that tests for an instance by completeness, as beman.transpose's `Monoid` does, is a third encoding. The model covers only its verdicts: they are a lookup's, at the same uses, under a third rule where the test's answer changes, a satisfaction result that differs at two points ([temp.constr.atomic]/3). The member overload set of an instance is not resolved: the model has no class member lookup, no using-declarations and no explicit object parameters. It takes the member call's answer as a parameter, and proves only what the operation object makes of it. Which members are accurate comes from compiling against beman.transpose, not from the model. Function parameter packs are outside the fragment. Applicative's `invoke` takes one after the argument that keys the lookup, and the pack does not enter the lookup.

12 Implementation experience

The model and the generated tests are in the overload repository. The Lean sources are under `Overload/`, built with Lean 4.35 and Batteries, and every result named here is audited to depend on no unproven axiom. `make differential` emits the 91 C++ programs and compiles each with the listed compilers; `differential/known-disagreements.txt` lists each case and compiler where an implementation departs from the text.

The main results, by file:

- `Overload/Typeclass.lean`: `coherence`, `stability`, `fmapObject_deduce`, `call_stability`, and the explicit-instance routes.
- `Overload/Separability.lean`: `separability`, `rank_beats_refinement`.
- `Overload/Inheritance.lean`: `inheritance_erasure`, `derived_class_deduction`, `superclass_identity`.
- `Overload/Access.lean`: `protected_base_selected_ill_formed`, `protected_base_deduced_ill_formed`.
- `Overload/Members.lean`: `fmapObject_member`, `call_stability_arity`, `arity_order_matters`, `registry_verdicts`.
- `Overload/Units.lean`: `verdicts_single`, `operation_object_not_separated`, `byValue_separated`, `call_stability_unit`.
- `Overload/Mandates.lean`: `mandate_inertness`, `constraint_promotion`, `promotion_reorders`.
- `Overload/Sfinae.lean`: `sfinae_requires`, `sfinae_loses_subsumption`, `sfinae_unshielded`.
- `Overload/Dispatch.lean`: `dispatch_styles`, `dispatch_hard_condition`.
- `Overload/Generality.lean` and `Overload/Transitivity.lean`: soundness and generality of deduction, and transitivity of partial ordering.

The probes of beman.transpose's own typeclasses, one translation unit per call, are in `probes/library/`, with their results at the commit probed in `docs/probes.md`.

The encoding studied is a reduction of beman.transpose (`github.com/bemanproject/typeclass`), kept faithful to it in `src/smd/overload/encoding.hpp`.

13 References

[beman.transpose] beman.transpose: typeclasses for C++ as a library.
https://github.com/bemanproject/typeclass

[CWG2369] Agustin Bergé. 2017-10-09. Ordering between constraints and substitution.
https://wg21.link/cwg2369

[N5054] Thomas Köppe. 2026-07-16. Working Draft, Programming Languages — C++.
https://wg21.link/n5054

[P2308R1] S. Davis Herring. 2023-12-18. Template parameter initialization.
https://wg21.link/p2308r1

[P3606R2] Corentin Jabot, Younan Zhang. 2026-09-21. On Overload Resolution, Exact Matches and Clever Implementations.
https://wg21.link/p3606r2